Complete finite programs for high-rank symmetric markings

Human Compression project

Companion computational record

The following four programs are included in full. The primitive-catalogue program supplies required bounded inputs to the uniform converse. The core and six-letter programs validate fully written general proofs. The final program recovers the source's separate degree-eight example and validates the branching-presentation correction; it is not a premise of the all-parameter principal results. The Python and formal replay drivers are operational artifacts, not hidden mathematical programs. All execution reports keep these roles separate.

1 Required primitive-group and block-stabilizer inputs

```
# Exact finite catalogue input for the written primitive-rank bound.
# This uses the GAP primitive-groups catalogue and complete
 ↪  maximal-subgroup
# algorithms. It does NOT search for C-group markings.
SetInfoLevel(InfoWarning,0);;
Need:=function(b,s) if not b then Error(s); fi; end;;
PrimeLength:=n->Length(FactorsInt(n));;
PrimitiveExceptions:=[];; total:=0;;
for n in [9..25] do
 for k in [1..NrPrimitiveGroups(n)] do
  g:=PrimitiveGroup(n,k);; total:=total+1;;
  if Size(g)<Factorial(n)/2 and PrimeLength(Size(g))>n-4 then
   Add(PrimitiveExceptions,[n,k,Size(g),PrimeLength(Size(g))]);
  fi;
 od;
od;
expected:=[[9,5,144,6],[9,6,216,6],[9,7,432,7],[9,8,504,6],
[9,9,1512,7],[10,4,720,7],[10,5,720,7],[10,6,720,7],
[10,7,1440,8],[11,6,7920,8],[12,2,95040,11],[16,11,322560,14]];;
Need(PrimitiveExceptions=expected,"primitive exception list changed");;
Print("PRIMITIVE_CATALOGUE ",total," ",PrimitiveExceptions,"\n");
for k in [5,6,7] do
 g:=PrimitiveGroup(9,k);; v:=Socle(g);;
 Need(Size(v)=9 and IsElementaryAbelian(v),"affine9 socle");
 q:=FactorGroup(g,v);;
 if k=7 then
  f:=FrattiniSubgroup(q);;
  Need(Size(f)=2,"GL2(3) Frattini order");
  Need(IsomorphismGroups(FactorGroup(q,f),SymmetricGroup(4))<>fail,"Fratt
   ↪  ini quotient S4");
  bound:=4; # mu(q)=mu(S4)<=3, then translation contribution1
 else bound:=PrimeLength(Size(q))+1; fi;
 Need(bound<=5,"affine9 bound");
 Print("AFFINE9 ",k," quotient=",Size(q)," bound=",bound,"\n");
od;
g:=PrimitiveGroup(9,9);; inv:=Filtered(AsList(g),x->Order(x)=2);;
```

```
Need(Size(Group(inv))=504,"odd extension involutions");;
Print("ODD_EXTENSION_INVOLUTION_SUBGROUP 504\n");
for k in [4,6,7] do
 g:=PrimitiveGroup(10,k);; s:=Socle(g);;
 Need(IsomorphismGroups(s,AlternatingGroup(6))<>fail,"degree10 A6 socle");
 Need(Index(g,s) in [2,4],"degree10 quotient");
 Print("DEGREE10 ",k," A6 quotient=",Index(g,s),"
  ↪  bound=",4+PrimeLength(Index(g,s)),"\n");
od;
Need(IsomorphismGroups(PrimitiveGroup(10,5),SymmetricGroup(6))<>fail,"deg
 ↪  ree10 S6");;
g:=PrimitiveGroup(16,11);; v:=Socle(g);;
Need(Size(v)=16 and IsElementaryAbelian(v),"affine16 socle");;
Need(IsomorphismGroups(FactorGroup(g,v),AlternatingGroup(8))<>fail,"affin
 ↪  e16 A8");;
Print("AFFINE16_BOUND 7\n");
for nk in [[9,8],[11,6],[12,2]] do
 g:=PrimitiveGroup(nk[1],nk[2]);; maxs:=MaximalSubgroupClassReps(g);;
 if nk=[9,8] then
  Need(ForAll(maxs,h->PrimeLength(Size(h))<=4),"PSL2(8) maximal bound");
  Print("PSL28_MAXIMAL_ORDERS ",SortedList(List(maxs,Size))," bound=5\n");
 elif nk=[11,6] then
  for h in maxs do
   inv:=Filtered(AsList(h),x->Order(x)=2);;
   j:=Group(inv);;
   Need(PrimeLength(Size(j))<=6,"M11 involution generated maximal bound");
  od;
  Print("M11_MAXIMAL_ORDERS ",SortedList(List(maxs,Size))," bound=7\n");
 else
  for h in maxs do
   if Size(h)=7920 then
    Need(IsomorphismGroups(h,PrimitiveGroup(11,6))<>fail,"M12 M11
     ↪  maximal");
   elif Size(h)=1440 then
    s:=Socle(h);;
    Need(IsomorphismGroups(s,AlternatingGroup(6))<>fail and
     ↪  Index(h,s)=4,"M12 A6 quotient maximal");
   else Need(PrimeLength(Size(h))<=7,"M12 remaining maximal bound");fi;
  od;
  Print("M12_MAXIMAL_ORDERS ",SortedList(List(maxs,Size))," bound=8\n");
 fi;
od;
# The finite numerical part of the Maroti envelope, before the analytic
# power-of-two argument valid from degree64 onwards.
for n in [26..63] do
 k:=LogInt(n,2);; b:=n*Product([0..k-1],i->n-2^i);;
 Need(b<2^(n-3),"integer primitive chain envelope");
od;
Print("MAROTI_INTEGER_ENVELOPE 26..63 PASS\n");
Print("PASS exact finite primitive inputs; no marking enumeration\n");
# The stronger independent-family input needed for primitive block
 ↪  stabilizers.
# The single exception to mu'(L)<=degree-3 is the natural degree6 copy of
 ↪  S5.
independentExceptions:=[];;
for n in [5..25] do
 for k in [1..NrPrimitiveGroups(n)] do
  g:=PrimitiveGroup(n,k);;
```

```
  if Size(g)<Factorial(n)/2 and PrimeLength(Size(g))>n-3 then
   Add(independentExceptions,[n,k,Size(g)]);
  fi;
 od;
od;
Need(independentExceptions=[[5,3,20],[6,1,60],[6,2,120],[7,5,168],
[8,3,1344],[8,5,336],[9,7,432],[9,9,1512],[10,7,1440],
[12,2,95040],[16,11,322560]],"independent primitive exception list");;
Print("INDEPENDENT_PRIMITIVE_EXCEPTIONS ",independentExceptions,"\n");
g:=PrimitiveGroup(5,3);;v:=Socle(g);;
Need(Size(v)=5 and IsCyclic(FactorGroup(g,v)),"F20 cyclic quotient");;
Need(IsomorphismGroups(PrimitiveGroup(6,1),AlternatingGroup(5))<>fail,"de
 ↪  gree6 A5");;
Need(IsomorphismGroups(PrimitiveGroup(6,2),SymmetricGroup(5))<>fail,"degr
 ↪  ee6 S5");;
g:=PrimitiveGroup(7,5);;
for h in MaximalSubgroupClassReps(g) do
 Need(PrimeLength(Size(h))<=2 or
  ↪  IsomorphismGroups(h,SymmetricGroup(4))<>fail,"PSL32 maximal bound");
od;
g:=PrimitiveGroup(8,3);;v:=Socle(g);;
Need(Size(v)=8 and IsElementaryAbelian(v),"AGL32 translation");;
Need(IsomorphismGroups(FactorGroup(g,v),PrimitiveGroup(7,5))<>fail,"AGL32
 ↪  linear quotient");;
g:=PrimitiveGroup(8,5);;s:=Socle(g);;
Need(Index(g,s)=2 and
 ↪  IsomorphismGroups(s,PrimitiveGroup(7,5))<>fail,"PGL27 socle");;
g:=PrimitiveGroup(9,7);;q:=FactorGroup(g,Socle(g));;
Need(ForAll(MaximalSubgroupClassReps(q),h->PrimeLength(Size(h))<=4),"GL23
 ↪  proper subgroup bound");;
g:=PrimitiveGroup(9,9);;s:=Socle(g);;
Need(Index(g,s)=3 and
 ↪  IsomorphismGroups(s,PrimitiveGroup(9,8))<>fail,"PGL28 field
 ↪  extension");;
g:=PrimitiveGroup(12,2);;
Need(ForAll(MaximalSubgroupClassReps(g),h->PrimeLength(Size(h))<=8),"M12
 ↪  independent bound9");;
Print("PASS primitive block-stabilizer identifications and independent
 ↪  bounds\n");

QUIT;
```

2 Validation of the three finite attachment interfaces

```
# Exact finite hypotheses of the tree-attachment intersection theorem.
# The 156 identities validate explicit proofs on four or five letters.
SetAssertionLevel(1);
SubsetIndices:=Combinations([1,2]);;
Young:=function(blocks)
 local gens,b;
 gens:=[];
 for b in blocks do
  if Length(b)>1 then Append(gens,List([2..Length(b)],i->(b[1],b[i])));fi;
 od;
 if Length(gens)=0 then return Group(());fi;
 return Group(gens);
end;;
```

```
CheckCore:=function(name,marks,root)
 local H,parts,Y,S,T,hs,ht,merged,other,C,i,x,plus,n;
 H:=[];parts:=[];Y:=[];n:=0;
 for S in SubsetIndices do
  if Length(S)=0 then hs:=Group(());else hs:=Group(marks{S});fi;
  Add(H,hs);Add(parts,Orbits(hs,[1..4]));Add(Y,Young(parts[Length(parts)]
   ↪ ));
 od;
 for i in [1..Length(H)]do
  C:=First(parts[i],b->root in b);other:=Filtered(parts[i],b->not root in
   ↪ b);plus:=Young(Concatenation([Concatenation(C,[5])],other));
  if Group(Concatenation(GeneratorsOfGroup(H[i]),[(root,5)]))<>plus then
   ↪ Error(name," failed saturation");fi;
  for x in [1..Length(H)]do
   S:=Position(SubsetIndices,Intersection(SubsetIndices[i],SubsetIndices[
    ↪ x]));
   if Intersection(H[i],H[x])<>H[S]then Error(name," failed core IP");fi;
   if Intersection(H[i],Y[x])<>H[S]then Error(name," failed mixed core
    ↪ intersection");fi;
   if Intersection(Y[i],Y[x])<>Y[S]then Error(name," failed partition
    ↪ meet");fi;
   n:=n+3;
  od;
  n:=n+1;
 od;
 Print(name,": PASS finite core checks=",n," root=",root,"
  ↪ orders=",List(H,Size),"\n");
end;;
CheckCore("A",[(2,3),(1,2)(3,4)],4);
CheckCore("B",[(2,3),(1,2)(3,4)],3);
CheckCore("C",[(1,2)(3,4),(1,3)(2,4)],4);
QUIT;
```

3 Validation of the local splitting obstructions

```
#!/usr/bin/env python3
"""Exact bounded local intersection obstructions for the L040 converse.

No classification package is used: every subgroup is enumerated in S_6 by
permutation multiplication. The model list is the complete list specified
in the written fracture-graph proof, not a search over arbitrary markings.
"""
from __future__ import annotations
import argparse
import hashlib
import json
from collections import deque
from pathlib import Path

IDENTITY = tuple(range(6))

def need(ok, message):
    if not ok:
        raise ValueError(message)

def mul(a, b):
    return tuple(b[a[i]] for i in range(6))
```

```
def trans(a,b):
    p=list(IDENTITY);p[a-1],p[b-1]=p[b-1],p[a-1];return tuple(p)

def product(*pairs):
    p=IDENTITY
    for a,b in pairs:p=mul(p,trans(a,b))
    return p

def models():
    out=[]
    for core_kind, core in [
        ('path', [product((1,2),(4,5)),product((2,3),(4,5)),trans(3,4)]),
       ('star', [product((1,3),(4,5)),product((2,3),(4,5)),trans(3,4)])]:
        for vertex in range(1,6):
            for twist in range(2 if vertex<4 else 1):
              p=mul(trans(vertex,6),product((4,5)) if twist else IDENTITY)
              out.append((f'{core_kind}_v{vertex}_twist{twist}',core+[p]))
    out.append(('parallel_opposite', [product((1,2),(3,4)),product((3,4),
     ↪  (5,6)),trans(2,3),trans(4,5)]))
    out.append(('parallel_same', [product((1,2),(5,6)),product((3,4),(5,6
     ↪  )),trans(2,5),trans(4,5)]))
    need(len(out)==18,'exact local model count')
    return out

def subgroup(gens,mask):
    seen={IDENTITY:[]};queue=deque([IDENTITY])
    while queue:
        p=queue.popleft()
        for i,g in enumerate(gens):
            if mask>>i&1:
                h=mul(p,g)
                if h not in seen:seen[h]=seen[p]+[i];queue.append(h)
    return seen

def word(gens,seq):
    p=IDENTITY
    for i in seq:p=mul(p,gens[i])
    return p

def record(name,gens):
    groups=[subgroup(gens,i) for i in range(16)]
    for I in range(16):
        for J in range(I+1,16):
            bad=set(groups[I]).intersection(groups[J]).difference(groups[
             ↪  I&J])
            if bad:
                w=min(bad)
                return {'name':name,'generators':[list(g) for g in gens],
                        'left':I,'right':J,'witness':list(w),
                       'left_word':groups[I][w],'right_word':groups[J][w],
                        'common_subgroup_order':len(groups[I&J]),
                       'actual_intersection_order':len(set(groups[I]).int
                         ↪  ersection(groups[J]))}
    raise ValueError('model unexpectedly has IP: '+name)

def verify(records):
```

```
    expected=models();need(len(records)==len(expected),'missing local
     ↪ case')
    for row,(name,gens) in zip(records,expected):
        need(row['name']==name and row['generators']==[list(g) for g in
         ↪ gens], 'model substitution')
        I,J=row['left'],row['right'];w=tuple(row['witness'])
        need(0<=I<16 and 0<=J<16,'bad support')
        need(all(I>>i&1 for i in row['left_word']), 'left word support')
        need(all(J>>i&1 for i in row['right_word']), 'right word support')
        need(word(gens,row['left_word'])==w==word(gens,row['right_word']),
         ↪ 'word identity')
        common=subgroup(gens,I&J)
        need(w not in common,'intersection witness belongs to required
         ↪ intersection')
        need(row['common_subgroup_order']==len(common),'common order')
        left=subgroup(gens,I);right=subgroup(gens,J)
        need(row['actual_intersection_order']==len(set(left).intersection
         ↪ (right)), 'intersection order')
    return len(records)

def run():
    parser=argparse.ArgumentParser();parser.add_argument('--out',required
     ↪ =True);args=parser.parse_args()
    out=Path(args.out);need(not out.exists() and not
     ↪ out.is_symlink(),'output must be fresh');out.mkdir(parents=True)
    rows=[record(name,gens) for name,gens in models()];verify(rows)
    controls=[]
    import copy
    for label,mutate in [
      ('omit_case',lambda r:r.pop()),
      ('identity_witness',lambda r:r[0].update(witness=list(IDENTITY),lef
       ↪ t_word=[],right_word=[])),
      ('alter_marking',lambda
       ↪ r:r[0]['generators'].__setitem__(0,list(IDENTITY))),
      ('wrong_support',lambda r:r[0].update(left=0)),
      ('false_common_order',lambda
       ↪ r:r[0].update(common_subgroup_order=999))]:
        altered=copy.deepcopy(rows);mutate(altered)
        try:verify(altered)
        except (ValueError,IndexError):controls.append({'control':label,'
         ↪ rejected':True})
        else:raise ValueError('mutation accepted: '+label)
    (out/'witnesses.json').write_text(json.dumps(rows,indent=2)+'\n')
    report={'status':'PASS','cases':len(rows),'controls':controls,
            'scope':'18 exact six-letter local IP obstructions;
             ↪ completeness of the local reduction is proved in TeX',
            'script_sha256':hashlib.sha256(Path(__file__).read_bytes()).h
             ↪ exdigest()}
    (out/'report.json').write_text(json.dumps(report,indent=2)+'\n');prin
     ↪ t(json.dumps(report))
if __name__=='__main__':run()
```

4 Auxiliary degree-eight example and branching correction

```
# Exact auxiliary example printed before source Section6, plus branching
 ↪ correction.
```

```
# The all-parameter classification and geometry do not depend on this
 ↪ example.
SetInfoLevel(InfoWarning,0);;
Need:=function(b,s) if not b then Error(s); fi; end;;
r:=[(1,2),(1,2)(3,4),(1,2)(7,8),(1,2)(5,6),(1,3)(6,8),(1,8)(3,6)];;
g:=Group(r);;Need(g=SymmetricGroup(8),"literal source example group");;
sets:=Combinations([1..6]);;subs:=List(sets,s->Subgroup(g,r{s}));;
ip:=0;;for i in [1..64] do for j in [1..64] do
 Need(Intersection(subs[i],subs[j])=subs[Position(sets,Intersection(sets[
  ↪ i],sets[j]))],"example full IP");ip:=ip+1;
od;od;
pars:=List([1..6],i->Subgroup(g,r{Difference([1..6],[i])}));;
Need(IsTransitive(pars[1],[1..8]),"source transitive parabolic");;
Need(Size(pars[1])=1344,"source transitive parabolic order");;
checks:=0;;
# Product-set criterion: membership in Ha Hj is represented by the orbit
 ↪ of Ha
# under Hj on right cosets. A common orbit element must be in the orbit
 ↪ under
# the exact intersection of the selected Hj. Check every nonempty anchor
 ↪ choice.
for a in [1..6] do
 cs:=RightCosets(g,pars[a]);;base:=Position(cs,RightCoset(pars[a],()));;
 hom:=ActionHomomorphism(g,cs,OnRight);;
 for js in Combinations(Difference([1..6],[a])) do
  if Length(js)>=2 then
   left:=[1..Length(cs)];;h:=g;;
   for j in js do
    left:=Intersection(left,Set(Orbit(Image(hom,pars[j]),base)));;
    h:=Intersection(h,pars[j]);;
   od;
   right:=Set(Orbit(Image(hom,h),base));;
   Need(left=right,"source example flag transitivity");checks:=checks+1;
  fi;
 od;
od;
Print("BOUNDARY_N8 group40320 IP",ip," flag_products",checks,"
 ↪ transitive_parabolic1344 PASS\n");
r9:=[(2,3),(1,2)(3,4),(4,5),(5,6),(5,7),(6,8),(7,9)];;
Need(Group(r9)=SymmetricGroup(9),"branching example generates S9");;
Need(Order(r9[3]*r9[4])=3 and Order(r9[3]*r9[5])=3,"two noncommuting
 ↪ bridge neighbours");;
Need(Order(r9[2]*r9[3])=6,"bridge intrinsic order6");;
Need(Length(Filtered(r9,x->Order(r9[2]*x)=6))=1,"unique order6
 ↪ neighbour");;
Print("BRANCHING_LITERAL_CORRECTION two_bridge_neighbours_order3
 ↪ PASS\n");
QUIT;
```